SQL Procedure

A **stored procedure** is a named set of SQL statements saved in a database. You call it whenever you want to perform a task, and you can pass values into it through **parameters**.

Unlike a **trigger**, which runs automatically when a table changes, a **procedure runs when explicitly called**.

For example, this **MySQL procedure** records a student payment using the `payments` table from our previous example:

```
DELIMITER //

CREATE PROCEDURE record_payment (
    IN p_student_id INT,
    IN p_amount DECIMAL(10, 2)
)
BEGIN
    INSERT INTO payments (student_id, amount)
    VALUES (p_student_id, p_amount);
END//

DELIMITER ;
```

To record a **$200 payment for student 1**, call:

```
CALL record_payment(1, 200.00);
```

How it works:

- **`CREATE PROCEDURE record_payment`** creates a procedure named `record_payment`.
- **`IN`** identifies an input parameter—a value supplied by the caller.
- **`p_student_id`** receives the student ID.
- **`p_amount`** receives the payment amount.
- **`BEGIN / END`** groups the procedure's statements.
- **`CALL`** executes the procedure with the supplied values.

With the earlier `apply_student_payment` trigger installed, the sequence is:

```
CALL record_payment(1, 200.00)
              ↓
Procedure inserts a payment
              ↓
Trigger automatically reduces the student's balance
```

The procedure does not subtract the payment itself, because the existing trigger already performs that action.

Stored procedures are useful for grouping repeatable database tasks, accepting inputs, and controlling how operations are performed. Support and syntax vary by DBMS; **SQLite does not support SQL stored procedures like this**.

Dropping or Replacing Triggers and Procedures

You can change triggers and stored procedures after creating them on a MySQL server, provided you have the required permissions.

To change their SQL code, MySQL requires you to **drop and recreate them**.

For example:

```
DROP TRIGGER cap_course_tuition;
```

Returning a Value From a Procedure

In MySQL, use an **`OUT` parameter** to return a value from a stored procedure. For example, to return the grade of a student, the procedure takes the student ID, course ID, and semester as inputs and looks up the matching row in `enrollments`.

```
DELIMITER //

CREATE PROCEDURE get_student_grade (
    IN p_student_id INT,
    IN p_course_id INT,
    IN p_semester VARCHAR(20),
    OUT p_grade VARCHAR(5)
)
BEGIN
    SET p_grade = (
        SELECT e.grade
        FROM enrollments AS e
        WHERE e.student_id = p_student_id
          AND e.course_id = p_course_id
          AND e.semester = p_semester
    );
END//

DELIMITER ;
```

Pass a session variable as the fourth argument to receive the grade. You can then display or reuse it in the same connection.

```
CALL get_student_grade(1, 101, 'Fall 2026', @student_grade);
SELECT @student_grade AS grade;
```